

Architecture Summary & Roadmap: Web Page Analyzer

This document provides a high-level overview of the web page analyzer service, its core design, and a roadmap for future enhancements. This version reflects the production-grade features implemented in the system.

1. Current System Summary

The project is a high-performance, resilient, and observable Go backend service that analyzes web pages. It is built using a strict Hexagonal Architecture (Ports & Adapters), separating the system into four distinct layers: Domain, Application, Infrastructure, and Transport.

Key Features:

- **Dual API Endpoints: Synchronous** (/api/v1/analyzes) for immediate results and **Asynchronous** (/api/v1/analyzes/async) for "fire-and-forget" requests, the purpose of which is to email the results to the user.
- **Modern Architecture:** Built using Clean Architecture to decouple business logic (application) from implementation details (infrastructure).
- **Robust & Resilient:**
 - **Circuit Breaker:** Uses `sony/gobreaker` in the `infrastructure/fetcher` to prevent cascading failures when fetching external URLs.
 - **Worker Pool:** A concurrent worker pool in `infrastructure/linkchecker` provides fast, safe, and concurrent link validation.
- **Production-Ready:**
 - **Authentication:** (Ready to be enabled in the next phase) Token-based auth middleware.
 - **Rate Limiting:** Global rate limiting (`golang.org/x/time/rate`) to prevent abuse.
 - **Idempotency:** The synchronous endpoint uses an Idempotency-Key header and a cache to prevent duplicate POST requests.
 - **Configuration:** Configuration is loaded from `config.properties` using `viper`.
- **Fully Observable:**
 - **Structured Logging:** `slog` is used for structured, queryable JSON logging.

- **Prometheus Metrics:** Exposes a /metrics endpoint with request duration histograms and counters.
 - **Profiling:** Exposes /debug/pprof endpoints for live performance profiling.
- **Configurable:** All settings (timeouts, cache TTLs, etc.) are managed externally.
- **Common Error Handling:** Centralize error handling

2. Core Design Principles

The architecture is built on these fundamental software design principles:

- **Hexagonal Architecture:** The system's foundation. It enforces the Dependency Rule, where all dependencies flow inward (e.g., infrastructure depends on application interfaces, but application does not depend on infrastructure).
- **SOLID:**
 - **S - Single Responsibility Principle (SRP):** Each package has one clear responsibility (e.g., fetcher fetches, parser parses, handler handles HTTP).
 - **I - Interface Segregation Principle (ISP):** The analysis/service.go file defines a clean, minimal interface, hiding the implementation complexity.
 - **D - Dependency Inversion Principle (DIP):** The AnalysisService depends on abstract interfaces (like PageFetcher, LinkChecker, cache.Service), not on concrete types. These are injected in main.go.
- **Separation of Concerns (SoC):** Each layer is unaware of the internal workings of the others. The transport layer knows nothing about gobreaker, and the analysis service knows nothing about HTTP status codes.

3. Design Patterns Used

- **Dependency Injection (DI):** Used in cmd/server/main.go to "wire" the application. Concrete infrastructure adapters (http_fetcher, in_memory_cache) are instantiated and injected into services that rely on their interfaces.
- **Ports & Adapters (Hexagonal Architecture):** This is the practical application of Clean Architecture.
 - **Port:** The analysis/service.go interface.
 - **Adapter:** The analysis/service_impl.go implementation.

- **Middleware (Chain of Responsibility):** The `transport/http/middleware` package uses this to create a composable pipeline for `Logger`, `Metrics`, `RateLimit`, and `Idempotency`.
- **Repository Pattern:** Used for `infrastructure/cache`. An interface (`cache/service.go`) defines the contract, and a concrete type (`cache/in_memory.go`) provides the implementation.
- **Worker Pool Pattern:** Implemented in `infrastructure/linkchecker/checker.go`. A fixed number of goroutines (`maxWorkers`) consume links from a jobs channel, with a `sync.WaitGroup` to coordinate completion. Uses a fixed-size goroutine worker pool (CSP pattern) for checking links. Prevents crashing the service with thousands of links by avoiding the launch of thousands of concurrent goroutines.
- **Circuit Breaker Pattern:** Implemented in the `infrastructure/fetcher` (as explained above) to wrap external HTTP calls, isolating failures.
- **Asynchronous Request-Reply:** The `/analyzes/async` endpoint implies this pattern, where a request is accepted immediately (202 Accepted) and the result is processed by a worker in the background.
- **Monitor Pattern:** The `infrastructure/cache/in_memory.go` implementation uses a `sync.RWMutex` to guard the shared data map, ensuring safe concurrent access.
- **Factory Functions:** `NewInMemoryCache()`, `NewLinkChecker()`, etc., are used to encapsulate the creation and initialization of components.

4. Next Phase Developments

The current architecture is well-positioned for these enhancements:

- **1. Complete the Async Flow:**
 - **Persistent Queue:** Implement a robust message broker (e.g., RabbitMQ or Kafka) for the `infrastructure/queue` to replace any in-memory channel.
 - **Audit Database:** Implement a real database (e.g., MongoDB) for the worker to write audit logs to.
 - **Result Notification:** Implement a mechanism to notify the user when the async job is done (e.g., email).

- **2. Enhance Infrastructure:**
 - **Distributed Cache:** Replace `cache/in_memory.go` with `cache/redis.go` to implement a distributed cache for idempotency and analysis results.
 - **JWT Authentication:** Fully implement the `auth/service_impl.go` and `infrastructure/auth/jwt.go` to sign and validate JWTs, and enable the `authMiddleware`.
- **3. Expand Analysis Features:**
 - **Screenshot Service:** Add a new `infrastructure/screenshot` adapter (e.g., using `chromedp`) and add the feature to the `AnalysisService`.
 - **Performance Metrics:** Integrate Lighthouse as a new infrastructure component.
 - **Accessibility (WCAG) Checks:** Add a new parser to check for accessibility violations.
- **4. Improve Observability:**
 - **Distributed Tracing:** Implement OpenTelemetry to trace requests as they flow from the transport layer, through the analysis service, queue, and worker.
 - **Expand Metrics:** Add business-level metrics, such as `web_analyzer_analysis_duration_seconds` (in the `AnalysisService`) and `web_analyzer_links_checked_total` (in the `LinkChecker`).

4. Set up and configurations

By considering the better deployment practices, UI and backend are considered as two different services. Both services to be checkout.

1. **webpage-analyzer-frontend**
 - a. All the instructions are provided in <https://github.com/ruchiranet1/webpage-analyzer-frontend#web-page-analyzer-frontend>
 - b. This is running on port 3000.
2. **webpage-analyzer-service**
 - a. All the instructions are provided in <https://github.com/ruchiranet1/webpage-analyzer-service#web-page-analyzer-service>
 - b. This is running on port 8080

5. Assumptions

- There could be login forms, without form attribute also.